



Mowka
Grow with Confidence

RAG

Missing Link



Reranking: what it is, why it matters



Shubham Kansal

Founder | Mowka



<https://mowka.in/>

The missing link: basic vs production RAG

A minimal pipeline fetches chunks then feeds them to the LLM. To move toward production, you insert a **second-stage reranker**: it takes that candidate set and sorts it by how well each chunk answers the question. The model then sees only the most useful pieces.

→ Phase 1: Fetch — quick, wide net (vector or keyword search)

📁 Phase 2: Rerank — reorder by real relevance to the query

🚀 Phase 3: Generate — only top-ranked chunks go to the LLM

That middle phase is what separates a demo from a system you can ship.



What you gain from a rerank step

Initial fetch is tuned for speed and coverage. The set you get can still contain passages that only seem on-topic. A reranker evaluates genuine fit to the question so the strongest candidates float to the top.

Sharper input — The model receives passages that truly address the ask, not just lexically similar text.

Cleaner context — Tangential hits are demoted so the context window stays on point.

Lower token use — Fewer, higher-value tokens mean lower cost and latency.

Quality of context drives quality of output.



Five ways to rerank

Widely used options for that second stage. Each balances latency, relevance, spend, and operational load.

1. Cross-encoder — one model scores query-doc pairs together; high accuracy, higher latency.

2. RRF — fuse rankings from multiple retrievers with a simple formula; no neural model, very fast.

3. Cohere Rerank — hosted API; strong relevance, no infra to run.

4. ColBERT — late interaction over token embeddings; solid trade-off between speed and accuracy.

5. LLM-as-judge — an LLM scores each doc for relevance; highest accuracy, higher per-query cost.



Cross-encoder and RRF in practice

Two popular choices and where they shine.

Cross-encoder

One model consumes query and document together and outputs a single relevance score. Fit when relevance outweighs speed and you rerank a modest set (e.g. 20–100 candidates).

RRF

Merge results from several retrievers (e.g. BM25 and vector search) using a rank-fusion formula. No neural model — ideal for hybrid setups and fast iteration.



Cohere Rerank API — minimal snippet

Cohere's hosted rerank endpoint: you send a query and a list of documents and get back scores or a top-n list. No GPUs or model hosting — a good fit when you want strong reranking without running your own models.

```
import cohere
co = cohere.Client('YOUR_API_KEY')
results = co.rerank(
    query="your search query",
    documents=retrieved_chunks,
    top_n=5,
    model='rerank-english-v2.0'
)
```

Supply your candidates and top_n; feed the reranked list into your LLM context.



Wrap-up

A second-stage reranker is what moves you from toy RAG to something you can rely on in production.

- Fetch is fast but rough; reranking sharpens the set by relevance.
- Stronger context means fewer made-up answers and more grounded ones.
- Choose a method based on your constraints: relevance, latency, budget, ops.

Slot a rerank phase between fetch and generation — that's the link that makes RAG trustworthy.

